

持仓管理模块 设计方案

版本: v2.0 | 日期: 2026-05-25 | 项目: 3L交易系统 | 分支: 持仓管理

✓ 已纳入部署架构考量

一、背景与现状

1.1 现有持仓数据

`private/holdings.json` 存储当前持仓快照，结构如下：

```
{
  "update_date": "2026-05-23",
  "holdings": [
    { "name": "药明康德", "code": "603259", "ratio": 14.70, "direction": "创新药" },
  ],
  "cash_ratio": 9.16
}
```

1.2 现有基础设施

文件	作用	状态
<code>private/holdings.html</code>	只读持仓展示页（表格风格）	将移出 private/
<code>private/holdings.json</code>	持仓数据存储（用户数据，仓库不跟踪）	待扩展字段
<code>services/holdings_service.py</code>	持仓读写服务	需增加写入
<code>backend/api/holdings.py</code>	持仓API路由	需增加POST
<code>frontend/public/js/pages/top_gainers.js</code>	SVG饼图实现（可复用 <code>renderPie</code> 函数）	现有
<code>frontend/public/js/stock_card.js</code>	个股诊断卡渲染（参考卡片风格）	参考

二、需求摘要

- 个股管理 — 新增、编辑、删除持仓个股
- 个股展示 — 以卡片风格展示每只持仓，显示名称、代码、方向、仓位%、止损价（含自动计算跌幅%）及当前股价/涨跌幅
- 新增持仓弹窗 — 搜索股票 → 显示当前价 → 选择方向 → 输入仓位%和止损价 → 自动计算止损跌幅%
- 方向分布饼图 — 复用现有 SVG 饼图实现，展示各方向仓位占比
- 持仓评估 — 基于市场环境评估当前持仓结构
- 持仓建议 — 基于评估结果给出可操作的调仓建议
- 不追溯历史 — 只维护当前快照，不记录仓位变更历史
- 盘后维护 — 当前版本只做盘后维护，盘中实时暂不介入
- 页面移出 **private/** — `holdings.html` 作为正常前端页面，由构建系统处理
- 部署就绪 — 用户数据与代码分离，一键部署时清空持仓

三、部署架构 (与 private/ 分离)

3.1 核心原则：代码与用户数据分离

部署给他人时，需确保不含任何持仓数据。关键在于：

- 页面代码 — Holdings.html 从 `private/` 移到 `frontend/holdings.html`，走标准构建流程
- 用户数据 — `holdings.json` 仍存 `DATA_DIR/private/`，首次部署自动创建空持仓
- 仓库跟踪 — `.gitignore` 忽略 `private/holdings.json`（作者的数据不推入仓库）

3.2 目录结构变化

当前	改后
<code>private/holdings.html</code> （页面+脚本混合）	<code>frontend/holdings.html</code> + <code>frontend/css/pages/holdings.css</code>
<code>private/holdings.json</code> （数据+页面混合目录）	<code>data/3l/private/holdings.json</code> （仅数据）
<code>private/trades.json</code>	不变，仍在 <code>private/</code>

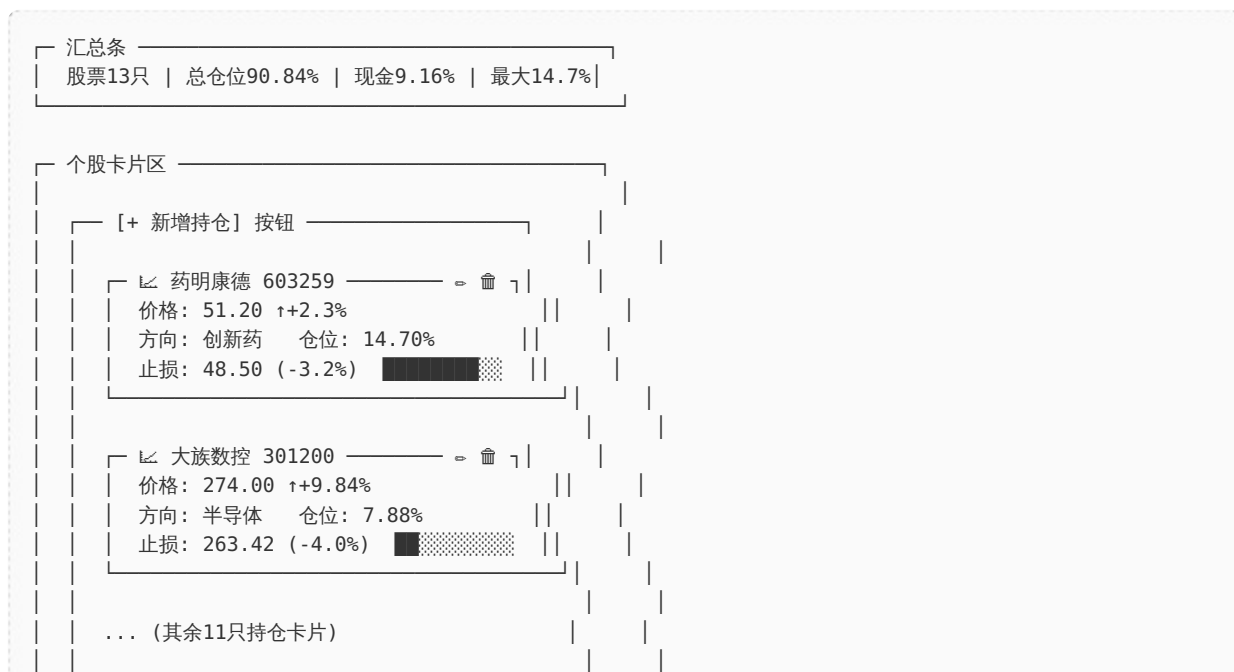
3.3 首次部署行为

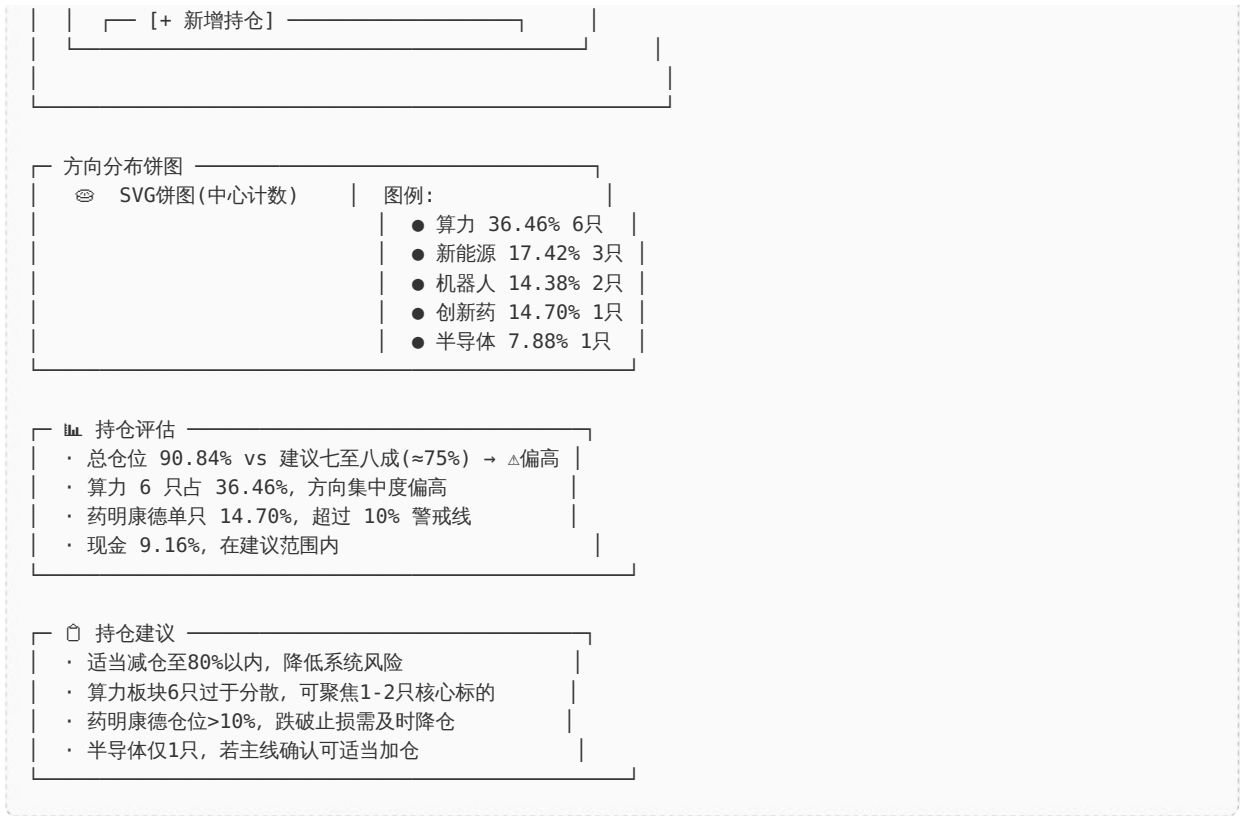
- 如果 `holdings.json` 不存在 → 自动创建 `{"holdings": [], "cash_ratio": 100}`
- 页面访问时如果持仓为空 → 显示 "暂无持仓" + 引导新增
- 无需额外初始化脚本

3.4 路径定义

- URL: `/holdings.html`
- 不加入导航栏 — 私密页面，仅通过书签/直接URL访问
- API 路径不变：`/api/holdings`（私密路径，数据不在前端HTML中硬编码）

四、页面布局





五、数据模型

5.1 holdings.json

```
{
  "update_date": "2026-05-25",
  "holdings": [
    {
      "name": "药明康德",
      "code": "603259",
      "ratio": 14.70,
      "direction": "创新药",
      "stop_loss_price": 48.50
    }
  ],
  "cash_ratio": 9.16
}
```

字段说明：

字段	类型	说明
name	string	股票名称
code	string	股票代码（6位）
ratio	number	仓位占比百分比
direction	string	所属方向（如"算力"、"新能源"）
stop_loss_price	number null	止损价（元），未设时为null

5.2 API 响应扩展 (GET /api/holdings)

后端返回时自动叠加各股的实时股价（腾讯接口），前端展示无需额外请求：

```
{
  "update_date": "2026-05-25",
  "holdings": [
    {
      "name": "药明康德", "code": "603259",
      "ratio": 14.70, "direction": "创新药",
      "stop_loss_price": 48.50,
      // ↓ 后端自动叠加的实时行情
      "price": 51.20,
      "change": 2.3,
      "stop_loss_pct": -3.2 // 从 stop_loss_price 和 price 计算
    }
  ],
  "cash_ratio": 9.16
}
```

六、API 设计

方法	路径	说明
GET	/api/holdings	获取持仓列表（含实时行情叠加）
POST	/api/holdings/save	保存持仓全量数据
GET	/api/search-stocks?q=xxx	搜索股票（已有，弹窗用）

6.1 POST /api/holdings/save

请求体：

```
{
  "holdings": [
    { "name": "药明康德", "code": "603259", "ratio": 14.70, "direction": "创新药", "stop_loss_price": 48.50 }
  ],
  "cash_ratio": 9.16
}
```

安全规则：

- 写入超过50只时，少于10只的写入会被拒绝（防误覆盖）
- 写入时自动更新 `update_date` 为当天日期
- cash_ratio 校验：必须 ≥ 0 且 ≤ 100

七、前端结构

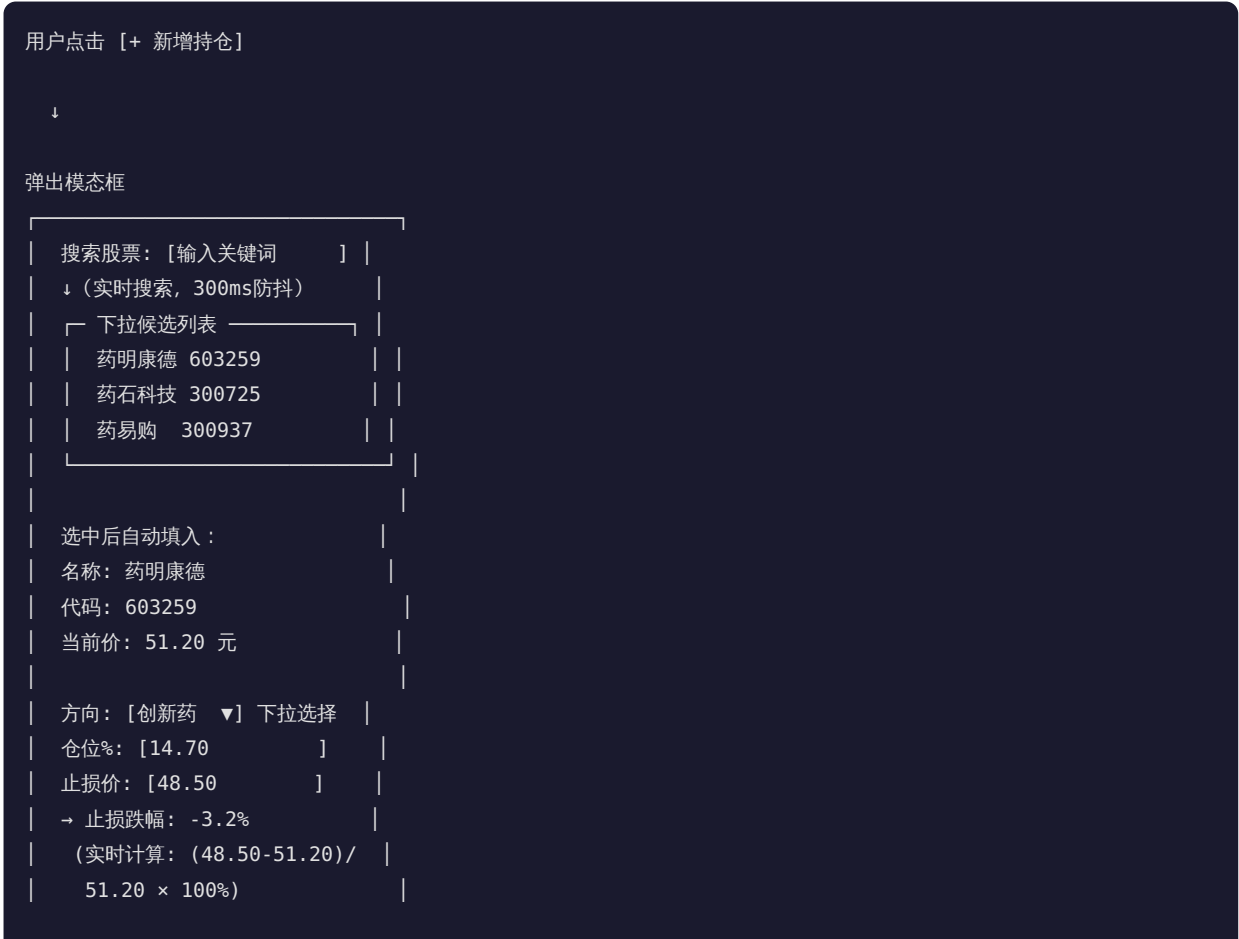
7.1 文件变更清单

文件	动作	说明
private/holdings.html	删除	旧只读表格页面，废弃
frontend/holdings.html	新建	持仓管理主页面（卡片风格+弹窗+饼图+评估+建议）
frontend/css/pages/holdings.css	新建	持仓页专用样式
frontend/build.py	更新	加入 holdings.html 的复制规则

7.2 页面功能分区

区域	功能
汇总条	显示股票数、总仓位、现金、最大单只
个股卡片列表	每只持仓一张卡片，含名称/代码/价格/方向/仓位/止损，右上角编辑和删除按钮
新增持仓按钮	卡片列表顶部和底部各一个，点击弹出新增弹窗
新增弹窗	搜索股票→选股→显示当前价→选方向→填仓位%→填止损价→自动算跌幅%→保存
编辑弹窗	同新增弹窗，但预填充已有数据
方向饼图	SVG饼图，中心显示总只数，右侧图例带数量和百分比
持仓评估	前端从API数据计算展示
持仓建议	前端从评估结果推导

7.3 新增/编辑弹窗流程图



[取消] [保存]

八、后端改动

8.1 services/holdings_service.py

函数	状态	说明
<code>get_holdings()</code>	已有	读取holdings.json
<code>get_holdings_with_prices()</code>	新增	读持仓 + 腾讯实时行情 → 叠加price/change/stop_loss_pct
<code>save_holdings(data)</code>	新增	写入holdings.json，更新update_date，防误覆盖检查
<code>get_trades()</code>	已有	不变

8.2 backend/api/holdings.py

路由	说明
<code>GET /api/holdings</code>	改为返回带实时行情的持仓（调用 <code>get_holdings_with_prices</code> ）
<code>POST /api/holdings/save</code> (新增)	接收JSON → 校验 → 写入 → 返回结果

8.3 相关但不受影响的模块

- `config.py` — HOLDINGS_PATH 和 TRADES_PATH 已存在，不动
- `generate_review_data.py` — 持仓数据来源不变（仍从 holdings.json 读），不需改动
- `direction_service.py` — 方向管理独立，弹窗读取方向列表
- `server.py` — 只要 holdings.html 被 build.py 复制到 dist/，server.py 会自动 serve 静态文件

九、持仓评估与建议逻辑

9.1 持仓评估指标

① 总仓位 vs 市场建议仓位

市场建议 来自 review_data.json 的 market.position_pct
(如"七至八成"映射为 70-80%)

实际总仓位 = 100% - cash_ratio

判定：实际 > 建议上限 → △偏高；实际 < 建议下限 → △偏低

② 单只集中度

单只仓位 > 10% → △单只集中

单只仓位 > 15% → ●严重集中

③ 方向集中度

单方向总仓位 > 30% → △方向集中

单方向总仓位 > 40% → 严重集中

④ 现金充裕度

现金 < 5% → 现金不足，可能无法灵活调仓

现金 > 30% → 现金过多，仓位利用率低

9.2 持仓建议生成

评估发现	对应建议
总仓位偏高	"建议适当减仓至X%以内，降低系统风险"
总仓位偏低	"可适当加仓，目前仓位偏低，市场环境支持"
单只 > 10%	"XX仓位超过10%，若跌破止损需及时降仓"
方向集中 > 30%	"XX方向X只占比过高，可聚焦1-2只核心标的"
现金偏少	"现金不足5%，建议预留部分现金应对调仓"
方向过度分散	"XX方向多只标的，建议精简持仓数量"
某方向偏少且为主线	"XX方向仅有1只，主线确认时可适当加仓"

十、详细执行计划

共 7 个步骤，按依赖顺序排列。每步完成后 git commit + 全回归。

步骤 1：后端 — 扩展 holdings_service

文件	改动
services/holdings_service.py	新增 save_holdings() + get_holdings_with_prices()

save_holdings(data):

1. 校验 data 含 holdings 列表 (list) 和 cash_ratio (number, 0-100)
2. 防误覆盖：如果当前 holdings.json 有 >50 只但请求写入 ≤10 只，拒绝
3. 更新 update_date 为当天
4. 原子写入（先写临时文件再 rename）
5. 返回 {"success": true, "count": N}

get_holdings_with_prices():

1. 读取 holdings.json
2. 批量调用腾讯实时行情接口获取各股当前价
3. 为每只持仓叠加 price、change、stop_loss_pct
4. 如果某只获取失败，price 和 change 用 null

步骤 2：后端 — 扩展 holdings API

文件	改动
----	----

POST /api/holdings/save:

1. 解析 JSON 请求体
2. 调用 save_holdings()
3. 处理异常 (JSON 解析失败、校验失败等)
4. 返回结果

步骤 3：后端 — 单元测试

文件	内容
tests/test_holdings_service.py	新建，覆盖 save + get + 防误覆盖

测试用例：

1. save_holdings 写入后 get_holdings 能正确读出
2. 防误覆盖：50只已有 → 写入5只 → 拒绝
3. 防误覆盖：50只已有 → 写入15只 → 允许
4. cash_ratio 超出 0-100 → 拒绝
5. 空持仓保存 → 正常 (首次部署场景)
6. update_date 自动更新为当天

步骤 4：前端 — 删除旧页面，创建新页面文件

动作	文件
删除	private/holdings.html (旧只读表格页)
新建	frontend/holdings.html — 页面骨架
新建	frontend/css/pages/holdings.css — 卡片+饼图+弹窗样式
更新	frontend/build.py — 加入 holdings.html + holdings.css 复制规则

页面脚本功能清单 (内联在 holdings.html)：

1. loadData() — GET /api/holdings → 渲染卡片 + 汇总条 + 饼图 + 评估 + 建议
2. renderCards(holdings) — 为每只持仓生成卡片 HTML (含实时价、仓位条、止损跌幅)
3. renderPie(directionData) — 复用 top_gainers.js 的 SVG 饼图逻辑
4. renderAssessment(data) — 计算并渲染持仓评估文本
5. renderSuggestions(assessment) — 基于评估结果生成建议文本
6. openAddModal() — 弹出新增弹窗
7. openEditModal(index) — 弹出编辑弹窗 (预填充数据)
8. deleteHolding(index) — 确认框 → 删除 → 重新计算 cash_ratio → save
9. saveHoldings() — POST /api/holdings/save → 刷新页面
10. searchStock(keyword) — 300ms 防抖 → GET /api/search-stocks?q= → 渲染下拉候选
11. recalcStopLossPct() — 止损价变化时自动计算跌幅%

步骤 5：前端 — 构建与部署

命令	说明
<code>python3 frontend/build.py</code>	npm build + 复制 dist/ 下的 holdings.html + holdings.css
<code>sudo systemctl restart 3l-server</code>	重启服务

步骤 6：验证

1. 访问 `http://43.136.177.133:8080/holdings.html` → 页面正常加载
2. 汇总条显示正确数字
3. 13只持仓卡片显示正确，含实时价格和涨跌幅
4. 编辑某只持仓仓位% → 保存 → 刷新后生效
5. 删除某只持仓 → 保存 → 刷新后消失
6. 新增持仓 → 搜索 → 选股 → 填方向/仓位/止损 → 保存 → 刷新后出现
7. 饼图各方向占比正确
8. 评估 & 建议显示合理
9. 全回归 319 测试通过

步骤 7：提交

提交	信息
commit 1	<code>feat: 新增持仓管理后端 - save_holdings + get_holdings_with_prices</code>
commit 2	<code>feat: 新增持仓管理前端 - 卡片/弹窗/饼图/评估/建议</code>
commit 3	<code>test: 持仓服务单元测试</code>
commit 4	<code>chore: 删除旧 private/holdings.html, 更新 build.py</code>

十一、备注与边界情况

- 防误覆盖：POST保存时，如果当前已有 >50 只但写入 <10 只，拒绝写入
- 止损价非必填：新增时可以不填止损价，此时 stop_loss_price 为 null
- 方向下拉：从 direction_service 读取已启用的方向列表
- 搜索接口：复用现有三层搜索（全量A股名→板块映射→板块名兜底）
- 实时行情：GET /api/holdings 调用腾讯接口，每次请求实时拉取
- 私有页面：holdings.html 不加入导航栏，仅通过私密URL访问
- 首次部署空数据：holdings.json 不存在时，get_holdings_with_prices 自动返回空持仓
- 仓库不跟踪用户数据：.gitignore 已忽略 `data/3l/private/`，持仓数据不会误提交